RÉPUBLIQUE TUNISIENNE

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

RAPPORT TECHNIQUE DE PROJET

Rapport final consolidé

Laboratoire de détection et d'analyse des cybermenaces

Prototype SOC/SOAR virtualisé

pfSense – Suricata – Wazuh – DVWA – Shuffle – AI SOC Analyst

Auteur	Thamer Bousbia
Plateforme	VMware Workstation
Réseau documenté	192.168.100.0/24
Période couverte	Septembre – octobre 2026
Date de consolidation	12 octobre 2026

Ce document fusionne le cadrage initial, le rapport de déploiement du 28 septembre 2026 et l'état de validation du 11 octobre 2026. Il intègre les preuves visuelles dans les chapitres concernés et distingue les résultats historiques des validations encore à refaire.

Résumé

Ce projet présente la conception, le déploiement et l'évaluation d'un laboratoire de cybersécurité virtualisé destiné à l'observation d'activités contrôlées et à l'étude d'une chaîne SOC/SOAR. Le laboratoire comprend un poste Kali, une cible Ubuntu exécutant DVWA, pfSense avec Suricata, un serveur Wazuh et une instance Shuffle. Une application Flask locale, AI SOC Analyst, traite séparément des exports JSON du SIEM.

Les documents de projet présentent d'abord une topologie isolée proposée. Le déploiement VMware décrit ensuite utilise le réseau `192.168.100.0/24` ; le plan initial `192.168.56.0/24` est repris comme étape de conception. Les tests documentés incluent la reconnaissance de la cible, la vérification de ses services Web, l'inspection de requêtes DVWA, des tentatives SQLi/XSS et des requêtes d'authentification répétées. Wazuh affiche plusieurs événements associés à ces scénarios. Une connexion SSH réussie a été journalisée, et Wireshark a servi à observer les échanges TCP/SSH et Web. AI SOC Analyst a importé un export comprenant 208 alertes et généré des rapports PDF.

Le rapport de déploiement du 28 septembre documente la réception historique d'alertes Wazuh par un webhook Shuffle. Toutefois, le diagnostic du 11 octobre indique une erreur « Failed getting org » dans Shuffle et ne confirme pas l'état opérationnel actuel. La chaîne Suricata → règle Wazuh 100500 → Shuffle et une action défensive automatisée ne sont pas démontrées de bout en bout dans l'état le plus récent. Le laboratoire est donc qualifié de prototype SOC/SOAR partiellement validé.

Mots-clés : cybersécurité, laboratoire virtualisé, SIEM, SOAR, Suricata, Wazuh, DVWA, Shuffle, détection.

Abstract

This report consolidates the project brief, the deployment report dated 28 September 2026, and the latest validation report dated 11 October 2026. It describes a virtual cybersecurity lab built with Kali Linux, an Ubuntu/DVWA target, pfSense and Suricata, Wazuh, Shuffle SOAR, and a local Flask application called AI SOC Analyst.

The documented work includes target reconnaissance, web-service checks, DVWA request inspection, controlled SQL injection and reflected XSS attempts, repeated DVWA authentication requests, successful SSH authentication, and packet inspection with Wireshark. Wazuh displayed alerts associated with several application tests. AI SOC Analyst processed a manually imported JSON export containing 208 alerts and generated PDF reports.

A Wazuh-to-Shuffle webhook was successfully exercised in the deployment phase. The latest report, however, records a "Failed getting org" error and does not confirm that the current workflow is operational. A real Suricata alert correlated through Wazuh rule 100500 to a current Shuffle execution, as well as any automated defensive action, remains unverified. The result is a promising but partially validated SOC/SOAR prototype.

Keywords: cybersecurity lab, SIEM, SOAR, Suricata, Wazuh, DVWA, Shuffle, detection.

Table des matières

Résumé i

Abstract ii

Liste des sigles et abréviations viii

1 Introduction générale et cadrage 1

1.1 Contexte . . . 1

1.2 Problématique . . . 1

1.3 Objectif général . . . 1

1.4 Objectifs spécifiques . . . 1

1.5 Périmètre et règles de sécurité . . . 2

1.6 Démarche et organisation . . . 2

1.7 Structure du rapport . . . 3

2 Conception et architecture du laboratoire 4

2.1 Évolution de la topologie . . . 4

2.2 Rôle des composants . . . 5

2.2.1 Kali Linux et DVWA . . . 5

2.2.2 pfSense et Suricata . . . 6

2.2.3 Wazuh . . . 6

2.2.4 Shuffle et application d'analyse . . . 6

2.3 Flux d'événements attendu . . . 6

2.4 Outils d'observation . . . 6

3 Déploiement et intégration technique 7

3.1 Environnement de virtualisation . . . 7

3.2 Configuration pfSense . . . 7

3.3 Suricata et journalisation réseau 8
3.4 Réception et décodage dans Wazuh 9
3.5 Événements hôte et tableau de bord 10

4 Campagne de tests et résultats 14
4.1 Reconnaissance et services exposés 14
4.2 Accès à DVWA et observation HTTP 15
4.3 Essais applicatifs et alertes Wazuh 17
4.4 Scénario SSH et analyse de trafic 21
4.5 Scénarios prévus mais non démontrés 23

5 Orchestration avec Shuffle SOAR 24
5.1 Déploiement décrit 24
5.2 Workflow et test du webhook 24
5.3 État le plus récent et réserve de validation 27
5.4 Cible fonctionnelle recommandée 27

6 Application AI SOC Analyst 29
6.1 Fonctionnement observé 29
6.2 Limites d'interprétation 32

7 Évaluation globale et recommandations 33
7.1 Bilan de validation 33
7.2 Limites et risques 34
7.3 Plan d'action priorisé 34
7.4 Protocole de validation reproductible 35

Conclusion générale 37

Sources documentaires 38

A Correspondance entre rapports sources et rapport consolidé 39

B Matrice des critères de réception 41

C Répertoire et conservation des preuves 42

Table des figures

2.1 Schéma de l'architecture initialement proposée 5
2.2 Vue générale d'un élément du dossier de mise en place 5

3.1 Console de configuration du laboratoire 7
3.2 Interface LAN surveillée par Suricata 8
3.3 Paramètres de journalisation Suricata 8
3.4 Événements EVE JSON visibles dans pfSense 9
3.5 Vue d'ensemble du tableau de bord Wazuh 10
3.6 Volume d'événements affiché dans Wazuh 11
3.7 Liste de résultats dans Wazuh 11
3.8 Résultats de recherche Wazuh 11
3.9 Vue détaillée d'événements dans Wazuh 12
3.10 Diagnostic en terminal pendant les essais 12
3.11 Suivi des événements depuis Wazuh 12
3.12 Vue de journaux courants 13

4.1 Reconnaissance de la cible avec Nmap 14
4.2 Vérification des ports Web avec Netcat 15
4.3 Page Apache observée sur la cible 15
4.4 Page d'accueil de DVWA 16
4.5 Navigation dans les modules DVWA 16
4.6 Vue complémentaire de l'application DVWA 16
4.7 Interface Burp Suite Community 17
4.8 Historique de requêtes dans Burp Suite 17
4.9 Requête de test SQL Injection 18
4.10 Événements Wazuh associés au scénario SQL 19

4.11 Requête de test XSS réfléchi . . . 19
4.12 Événements Wazuh associés au scénario XSS . . . 19
4.13 Requêtes d'authentification répétées sur DVWA . . . 20
4.14 Alerte Wazuh sur l'activité répétée . . . 20
4.15 Recherche des alertes dans Wazuh . . . 20
4.16 Terminal associé au scénario SSH . . . 21
4.17 Journal SSH de la cible . . . 21
4.18 Commandes et échanges liés à SSH . . . 22
4.19 Authentification SSH visible dans Wazuh . . . 22
4.20 Capture Wireshark du trafic SSH . . . 22
4.21 Capture Wireshark du trafic HTTP . . . 23

5.1 Applications disponibles dans Shuffle . . . 24
5.2 Workflow de test Wazuh vers Shuffle . . . 25
5.3 État non initialisé du webhook . . . 25
5.4 Webhook en état de fonctionnement lors d'un test . . . 26
5.5 Action de retour configurée dans Shuffle . . . 26
5.6 Historique d'exécutions du workflow . . . 26
5.7 Détail d'une exécution et de son payload . . . 27

6.1 Interface locale AI SOC Analyst . . . 29
6.2 Sélection du fichier à analyser . . . 30
6.3 Console de l'application Flask . . . 30
6.4 Confirmation du traitement des alertes . . . 30
6.5 Exemple de rapport généré . . . 31
6.6 Rapport complémentaire avec statistiques . . . 31
6.7 Vue complémentaire du rapport d'analyse . . . 31

Liste des tableaux

2.1 Inventaire consolidé des composants et de leur rôle 4

3.1 Éléments de configuration réseau rapportés le 28 septembre 2026 9
3.2 Exemples de règles Wazuh visibles dans le rapport de déploiement 10

4.1 Scénarios DVWA et portée des résultats rapportés 18

7.1 Matrice de validation consolidée au 12 octobre 2026 33
7.2 Travaux recommandés et preuves attendues 35

A.1 Traçabilité des principaux apports des trois rapports 39

B.1 Critères de réception proposés 41

C.1 Éléments à archiver selon le type de validation 42

Liste des sigles et abréviations

API	Application Programming Interface
DVWA	Damn Vulnerable Web Application
EVE	Extensible Event Format, format JSON de journalisation de Suricata
FIM	File Integrity Monitoring, surveillance de l'intégrité des fichiers
HTTP	Hypertext Transfer Protocol
HTTPS	HTTP protégé par TLS
IDS / IPS	Intrusion Detection System / Intrusion Prevention System
JSON	JavaScript Object Notation
LAN / WAN	Local Area Network / Wide Area Network
PCAP	Packet Capture, fichier de capture réseau
SIEM	Security Information and Event Management
SOAR	Security Orchestration, Automation and Response
SOC	Security Operations Center
SQLi	SQL Injection
SSH	Secure Shell
VM	Machine virtuelle
XSS	Cross-Site Scripting

Chapitre 1

Introduction générale et cadrage

1.1 Contexte

Les environnements de sécurité doivent pouvoir observer les activités réseau et système, rassembler les traces pertinentes et permettre aux analystes d'évaluer les événements avant de décider d'une réponse. Le projet étudie ces fonctions à petite échelle dans un réseau virtualisé, où les événements peuvent être générés volontairement et les preuves conservées sans viser des systèmes externes.

Le travail a évolué en trois temps documentaires. Le premier rapport définit le besoin, les scénarios et une architecture proposée. Le second décrit l'installation effective sous VMware Workstation et une première validation du transport Wazuh vers Shuffle. Le troisième résume les tests DVWA, SSH et les outils d'analyse, puis relève des intégrations à revalider. Ces documents sont fusionnés ici dans l'ordre logique du projet, et non reproduits à la suite.

1.2 Problématique

La problématique est la suivante : comment établir, dans un laboratoire contrôlé, une chaîne qui transforme une activité de test autorisée en traces réseau et hôte exploitables, en alertes compréhensibles et en éléments d'orchestration vérifiables, tout en séparant clairement une validation observée d'une fonctionnalité simplement configurée ou prévue ?

1.3 Objectif général

Concevoir et évaluer un laboratoire SOC/SOAR virtualisé permettant d'observer des scénarios pédagogiques, de collecter les événements au moyen de Wazuh, de tester l'inspection réseau Suricata et d'étudier le transfert d'alertes vers Shuffle.

1.4 Objectifs spécifiques

— Définir une topologie isolée séparant le poste de test, le pare-feu/IDS, le SIEM, la cible Web et l'orchestrateur.

— Installer et décrire les principaux composants et leur rôle dans la chaîne d'observation.

— Réaliser des vérifications de services et des tests Web contrôlés sur la seule cible DVWA du laboratoire.

— Comparer les traces réseau, les journaux de la cible et les alertes Wazuh associées aux scénarios.

— Documenter l'intégration historique Wazuh–Shuffle, puis qualifier son état ultérieur sans assimiler une ancienne réussite à un fonctionnement actuel.

— Examiner l'import manuel de données Wazuh dans AI SOC Analyst et préciser les limites du mode statistique et du mode IA.

1.5 Périmètre et règles de sécurité

Les essais rapportés concernent les machines virtuelles du réseau privé du projet. DVWA est une application intentionnellement vulnérable utilisée à des fins pédagogiques. Les résultats ne caractérisent pas la sécurité d'un service réel. Toute répétition doit rester limitée aux hôtes autorisés, au réseau isolé et à des scénarios proportionnés.

Le document ne prétend pas qu'une exploitation a réussi lorsque la preuve n'établit qu'une requête, une redirection ou une alerte. En particulier, une réponse HTTP 302 ne prouve pas une extraction SQL, une capture Burp ne démontre pas l'exécution d'un XSS, et des requêtes répétées ne prouvent pas la compromission d'un compte.

1.6 Démarche et organisation

Le plan initial répartissait le projet sur six semaines : conception de la topologie et des scénarios ; installation ; reconnaissance et premières captures ; exécution des tests ; amélioration de la corrélation et des mesures de mitigation ; puis livraison du rapport et démonstration. Il s'agit du calendrier prévu dans le cahier des charges, pas d'une affirmation que chaque étape a été achevée à la date prévue. **[R1, pp. 2–5]**

La méthode de vérification appliquée à la lecture des preuves suit les étapes ci-dessous :

1. identifier l'hôte, l'adresse, le scénario et la configuration en vigueur ;
2. distinguer les objectifs planifiés, les configurations installées et les résultats effectivement observés ;
3. rapprocher l'horodatage d'une action de la capture, du journal source et de l'alerte SIEM ;
4. interpréter chaque résultat à la lumière de ce qu'il prouve réellement ;
5. revalider les composants dont l'état a pu changer après une reconfiguration.

1.7 Structure du rapport

Le chapitre 2 présente l'architecture et l'évolution des adresses. Le chapitre 3 rassemble la mise en place de pfSense, Suricata et Wazuh. Le chapitre 4 analyse les tests réseau, Web et système. Le chapitre 5 décrit Shuffle et les limites de l'orchestration actuelle. Le chapitre 6 traite AI SOC Analyst. Le chapitre 7 consolide les résultats, les risques et les actions recommandées avant la conclusion générale.

Chapitre 2

Conception et architecture du laboratoire

2.1 Évolution de la topologie

Le document de cadrage proposait un LAN isolé en `192.168.56.0/24`, avec des adresses d'exemple : pfSense en .1, Kali en .10, Wazuh en .20 et la cible en .30. Les rapports techniques ultérieurs décrivent le laboratoire réellement déployé sur `192.168.100.0/24`. Les deux plans correspondent à des étapes différentes et ne doivent pas être confondus. Dans la suite du rapport, le réseau `192.168.100.0/24` désigne l'environnement documenté lors du déploiement et des essais récents. **[R1, p. 2 ; R2, pp. 3–5 ; R3, p. 2]**

TABLE 2.1 – Inventaire consolidé des composants et de leur rôle

Composant	**Adresse / système**	**Rôle et état décrit dans les sources**
pfSense + Suricata	192.168.100.254	Passerelle LAN et inspection IDS. pfSense 2.8.1-RELEASE ; interface WAN observée en DHCP sur 192.168.62.128/24.
Wazuh	192.168.100.20	Manager, tableau de bord SIEM et réception des événements système/syslog.
Ubuntu DVWA	192.168.100.30	Ubuntu 22.04, Apache, application DVWA et agent Wazuh 4.14.8.
Shuffle SOAR	192.168.100.40	VM initialement prévue pour DFIR-IRIS, réaffectée au déploiement Shuffle par Docker Compose. Accès à revalider selon le rapport le plus récent.
Kali Linux	192.168.100.100	Poste de génération du trafic contrôlé, de reconnaissance et d'essais Web.
AI SOC Analyst	127.0.0.1 :5000	Application Flask locale sur la machine de développement ; import manuel d'un export Wazuh.
VM hors chaîne	Non précisée	ubuntu-devsec et Keycloak-Lab figurent dans l'inventaire sans rôle direct établi dans la chaîne testée.

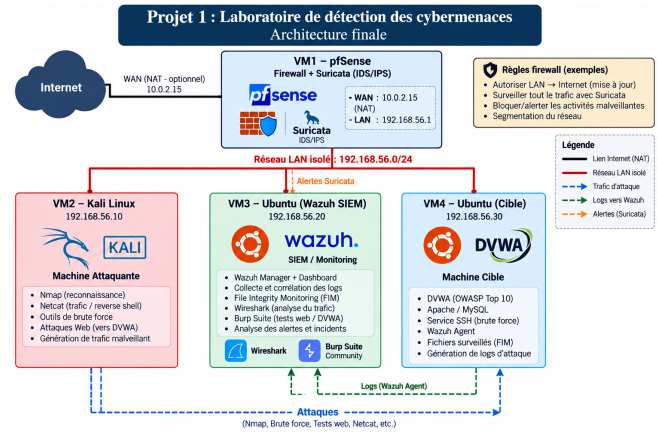


FIGURE 2.1 – Schéma de l'architecture initialement proposée

Lecture de la preuve : Le schéma montre les fonctions principales du laboratoire. Son plan IP appartient à la phase de conception ; les adresses de déploiement retenues dans ce rapport sont celles du réseau 192.168.100.0/24.



FIGURE 2.2 – Vue générale d'un élément du dossier de mise en place

Lecture de la preuve : Cette capture accompagne l'inventaire visuel des preuves de configuration. Elle est conservée comme trace de contexte et ne suffit pas, seule, à démontrer l'état actuel de tous les services.

2.2 Rôle des composants

2.2.1 Kali Linux et DVWA

Kali représente le poste de test et génère les connexions autorisées vers la cible. DVWA fournit des modules volontairement vulnérables, notamment des exercices de SQL Injection, de XSS réfléchi et d'authentification. La cible est décrite comme accessible sur le réseau interne à l'adresse `192.168.100.30/dvwa` ; elle ne doit pas être exposée à Internet. **[R1, pp. 2–3 ; R2, p. 9 ; R3, p. 2]**

2.2.2 pfSense et Suricata

pfSense assure le routage et le filtrage du laboratoire. Suricata est associé à l'interface LAN pour inspecter les flux en mode détection. L'architecture prévoit la journalisation structurée en EVE JSON et le transfert des événements vers Wazuh. La configuration de blocage était désactivée dans le rapport de déploiement, ce qui convient à une phase d'observation sans action de prévention automatique. **[R2, pp. 5–7]**

2.2.3 Wazuh

Wazuh centralise les journaux reçus de pfSense/Suricata et des agents installés sur les hôtes. Il permet de rechercher les événements, d'afficher les alertes et de réaliser de la surveillance d'intégrité (FIM) si elle est configurée. Les preuves confirment l'usage du tableau de bord et de règles de détection hôte ; elles ne démontrent pas que le scénario FIM initialement prévu a été exécuté avec succès. **[R1, p. 3 ; R2, pp. 7–10 ; R3, p. 3]**

2.2.4 Shuffle et application d'analyse

Shuffle est utilisé comme plateforme d'orchestration par webhook. L'application AI SOC Analyst suit un chemin distinct : elle reçoit un export JSON choisi manuellement par l'utilisateur et génère une sortie documentaire locale. Cette étape n'est pas une intégration temps réel du SIEM à l'application.

2.3 Flux d'événements attendu

La chaîne réseau visée est :

Test autorisé depuis Kali → inspection Suricata sur pfSense → transfert syslog/EVE → décodage et alerte Wazuh → webhook Shuffle → vérification d'une action sûre

En parallèle, l'agent Wazuh de DVWA collecte des événements hôte. La voie AI SOC Analyst est indépendante et implique un export puis un import manuel.

2.4 Outils d'observation

Nmap et Netcat sont employés pour la découverte et le contrôle de connectivité. Burp Suite Community permet d'observer les requêtes/réponses HTTP. Wireshark ou tcpdump servent à examiner les paquets et à conserver des PCAP. Wazuh sert à corréler les traces disponibles. Ces outils complètent les mécanismes de collecte ; aucun résultat provenant d'une seule interface ne suffit à démontrer toute la chaîne causale.

Chapitre 3

Déploiement et intégration technique

3.1 Environnement de virtualisation

Le déploiement décrit dans le deuxième rapport est hébergé sous VMware Workstation. Cinq machines participent directement à la chaîne de détection ; deux autres apparaissent dans l'inventaire mais sont hors périmètre. Shuffle a remplacé DFIR-IRIS sur la VM prévue à cet effet. Le système hôte et les configurations détaillées qui ne figurent pas dans les rapports ne sont pas supposés ici. **[R2, pp. 3–4]**



FIGURE 3.1 – Console de configuration du laboratoire

Lecture de la preuve : Cette capture de terminal documente une opération de mise en place ou de diagnostic. Elle est présentée comme élément de contexte, sans lui attribuer un résultat plus précis que ce que permet son contenu visuel.

3.2 Configuration pfSense

Le rapport du 28 septembre identifie pfSense 2.8.1-RELEASE (amd64). L'interface WAN (em0) recevait l'adresse `192.168.62.128/24` par DHCP. L'interface LAN (em1) utilisait l'adresse `192.168.100.254/24`. Suricata démarrait avec la VM selon la console décrite. Ces valeurs sont celles rapportées à cette date ; elles doivent être contrôlées sur la configuration actuelle avant toute répétition. **[R2, p. 5]**

3.3 Suricata et journalisation réseau

Suricata était activé sur l'interface LAN, avec l'identifiant « LAN IDS », détection de motifs en mode AUTO et blocage désactivé. Les règles ET Open étaient activées. **[R2, pp. 5–6]**

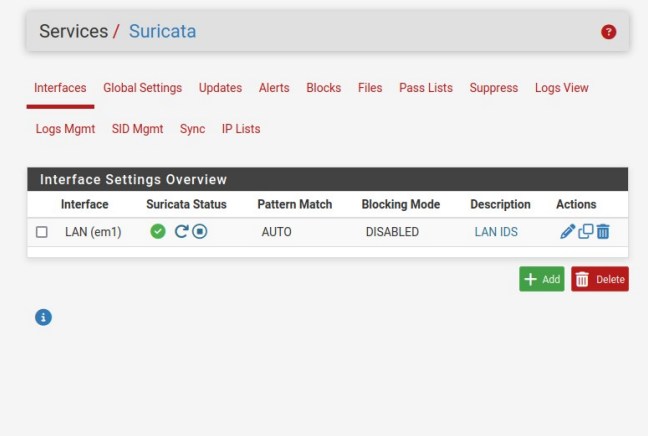


FIGURE 3.2 – Interface LAN surveillée par Suricata

Lecture de la preuve : Cette capture du rapport de déploiement (figure 4) montre l'interface em1 nommée « LAN IDS », le mode AUTO et le blocage désactivé. Elle atteste la configuration documentée à cette date, pas son état actuel.

Les alertes étaient configurées pour être journalisées vers le syslog de pfSense avec la facilité LOCAL1 et la priorité NOTICE ; le flux EVE JSON était conservé pour l'analyse structurée. **[R2, p. 6]**

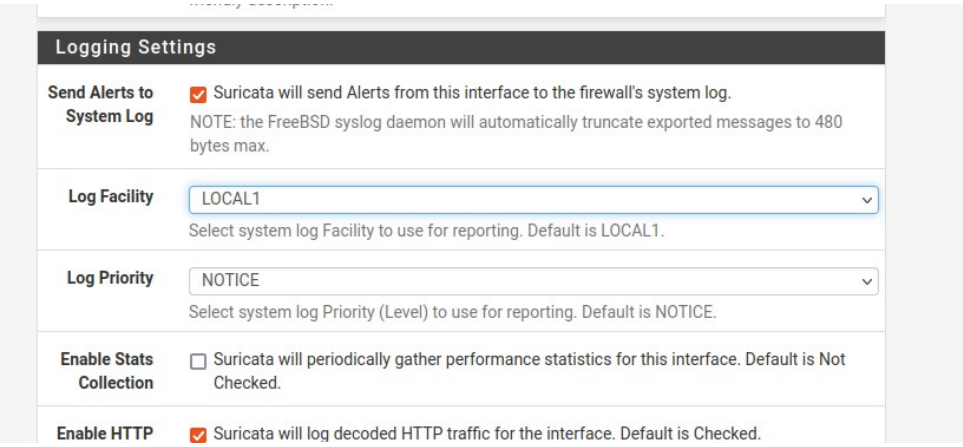


FIGURE 3.3 – Paramètres de journalisation Suricata

Lecture de la preuve : Cette capture du rapport de déploiement (figure 5) montre l'envoi des alertes vers le journal système de pfSense ainsi que les valeurs LOCAL1 et NOTICE.

Les journaux EVE visibles dans pfSense comprenaient notamment des événements de type `http` et `fileinfo` produits lors d'une navigation vers l'interface pfSense. Le rapport précise qu'il s'agissait d'événements réseau et non d'alertes de détection. Cette distinction est importante : la présence de télémétrie prouve que du trafic est observé, mais n'établit pas qu'une signature de menace ait été déclenchée.

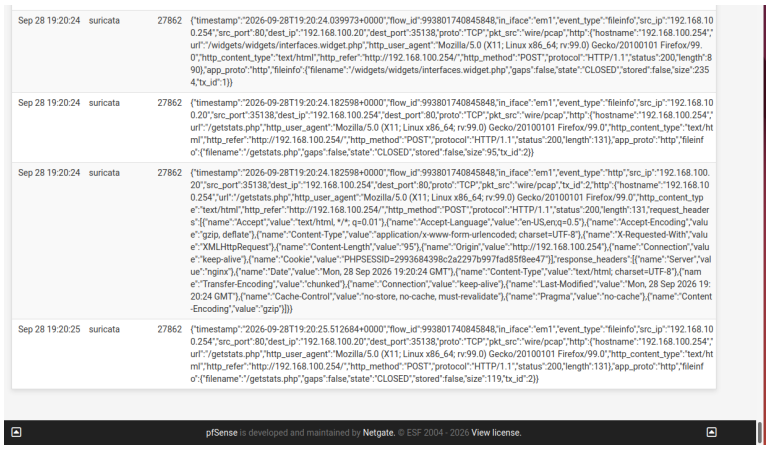


FIGURE 3.4 – Événements EVE JSON visibles dans pfSense

Lecture de la preuve : Cette capture du rapport de déploiement (figure 6) montre des événements HTTP et fileinfo générés pendant une navigation. Ce sont des journaux de trafic et non des alertes Suricata.

3.4 Réception et décodage dans Wazuh

Le manager Wazuh était configuré pour recevoir le syslog sur UDP/514. Une capture réseau sur la VM Wazuh a montré des paquets en provenance de `192.168.100.254:514` à destination de `192.168.100.20:514`. Le rapport du 28 septembre qualifie la réception syslog/EVE de validée à cette étape. Cette validation historique ne dispense pas de vérifier la configuration après toute modification ultérieure. **[R2, p. 7 ; R3, p. 3]**

Un décodeur local nommé `suricata-eve` et fondé sur le décodage JSON a été décrit. La règle locale 100500 (niveau 8) devait créer une alerte Wazuh lorsque le champ `event_type` valait `alert`, avec la signature Suricata dans sa description. Le rapport précise que le décodeur avait été essayé avec `wazuh-logtest`, mais que la production d'une alerte Suricata réelle par cette règle restait à confirmer. Le test du décodeur ne doit donc pas être présenté comme une alerte réseau de bout en bout. **[R2, p. 9 ; R3, p. 3]**

TABLE 3.1 – Éléments de configuration réseau rapportés le 28 septembre 2026

Élément	Valeur rapportée	Portée de la preuve
LAN pfSense	em1, 192.168.100.254/24	Interface de laboratoire utilisée pour l'inspection Suricata.
WAN pfSense	em0, DHCP, 192.168.62.128/24	Valeur observée lors du rapport de déploiement.
Transport vers Wazuh	syslog UDP/514	Réception de paquets vérifiée historiquement par capture réseau.
Décodage EVE	JSON, décodeur local	Décodage préparé et testé avec un outil de test Wazuh.
Règle Suricata	ID 100500, niveau 8	Règle préparée ; alerte issue d'un événement Suricata réel à revalider.

3.5 Événements hôte et tableau de bord

Le rapport de déploiement décrit des événements recueillis sur les agents osboxes, DVWA et DFIR-IRIS. Il rapporte 2 177 événements dans Threat Hunting sur une fenêtre de 24 heures à la date de la capture. Ce volume est un instantané historique, non un indicateur de performance actuel.

TABLE 3.2 – Exemples de règles Wazuh visibles dans le rapport de déploiement

Règle	Description rapportée	Niveau	Agent cité
5501 / 5502	Ouverture / fermeture de session PAM	3	osboxes, DVWA
5402	Sudo vers root réussi	3	osboxes
5710	Tentative SSH avec utilisateur inexistant	5	osboxes
2501	Échec d'authentification syslog	5	DFIR-IRIS
40704	Service systemd arrêté après une erreur	5	DVWA, osboxes
52002	AppArmor DENIED	3	DVWA
502 / 503	Démarrage du serveur ou d'un agent Wazuh	3	osboxes, DVWA

Le filtre sur la règle 5402 retournait neuf événements regroupés entre 07 :33 et 07 :51, attribués dans le rapport aux opérations d'administration de la VM Wazuh. Ces événements illustrent l'intérêt de conserver l'agent source et le contexte temporel afin de ne pas assimiler toute alerte à un scénario d'attaque. **[R2, pp. 7–9]**

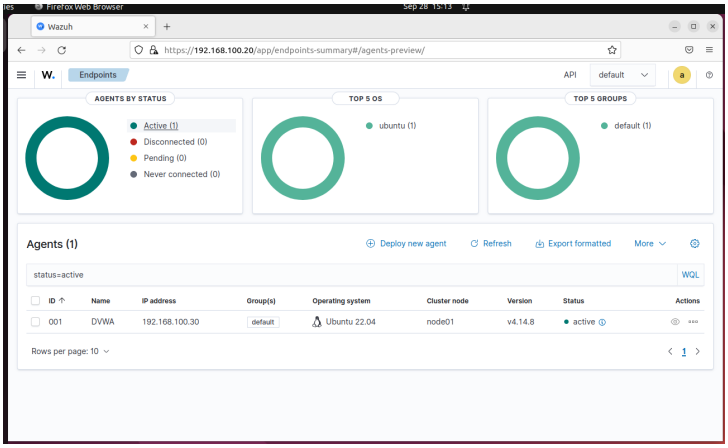


FIGURE 3.5 – Vue d'ensemble du tableau de bord Wazuh
Lecture de la preuve : Le tableau de bord présente des indicateurs agrégés consultés pendant la mise en place. Les compteurs varient dans le temps et ne constituent pas une mesure permanente de l'activité.

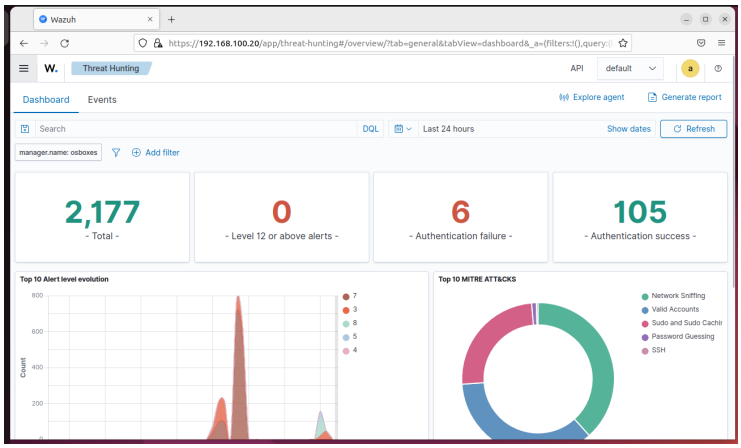


FIGURE 3.6 – Volume d'événements affiché dans Wazuh
Lecture de la preuve : Cette vue accompagne le relevé historique de 2 177 événements sur 24 heures rapporté pendant la phase de déploiement.

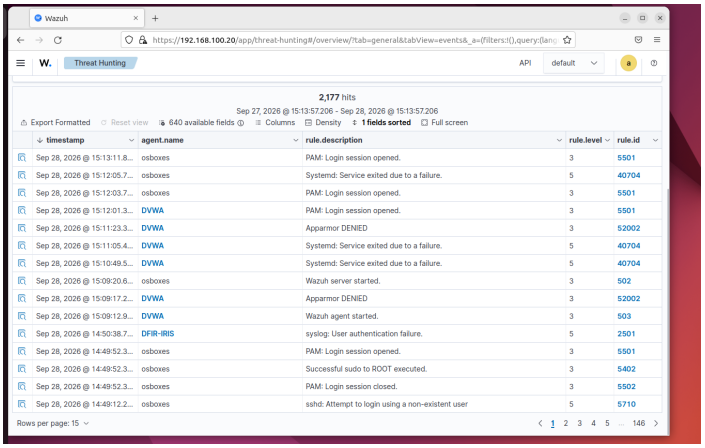


FIGURE 3.7 – Liste de résultats dans Wazuh
Lecture de la preuve : La capture montre des événements consultés dans le SIEM ; leur interprétation nécessite l'agent, l'horodatage, la règle et le journal source.

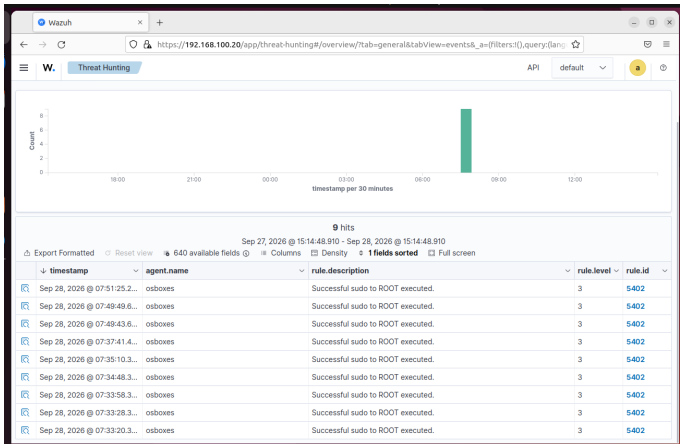


FIGURE 3.8 – Résultats de recherche Wazuh
Lecture de la preuve : Cette vue complémentaire illustre l'examen des événements du laboratoire et doit être lue dans le contexte du scénario associé.

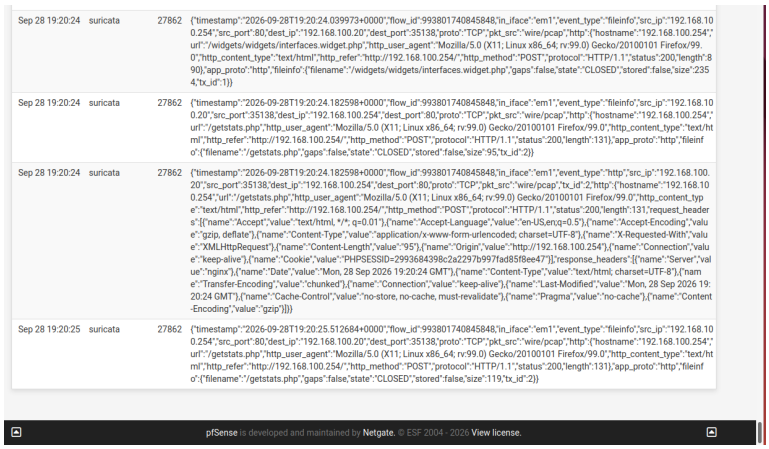


FIGURE 3.9 – Vue détaillée d'événements dans Wazuh
Lecture de la preuve : L'image conserve une trace visuelle de la consultation SIEM ; elle ne remplace pas l'export brut nécessaire à une analyse reproductible.



FIGURE 3.10 – Diagnostic en terminal pendant les essais
Lecture de la preuve : Cette capture de console est conservée comme preuve complémentaire des vérifications techniques ; le rapport ne lui attribue pas à elle seule un résultat de détection spécifique.

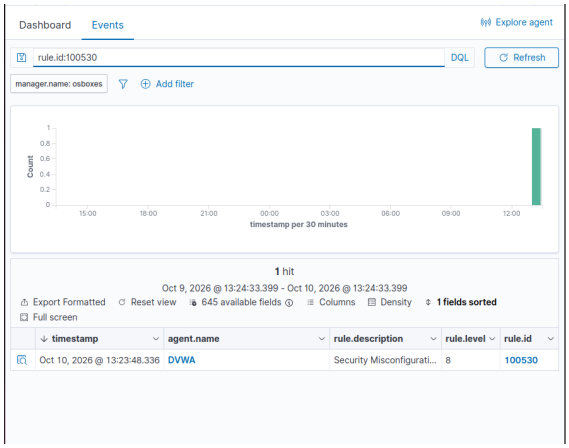
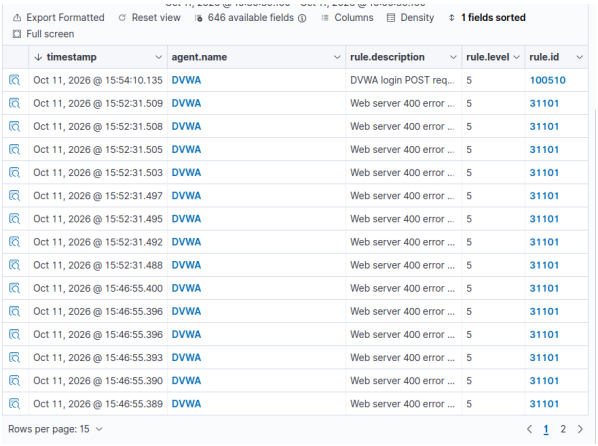


FIGURE 3.11 – Suivi des événements depuis Wazuh
Lecture de la preuve : La capture illustre le filtrage ou le suivi d'événements dans le SIEM ; une corrélation rigoureuse exige le rapprochement avec la source brute.



Export Formatted | Reset view | 646 available fields | Columns | Density | 1 fields sorted | Full screen

timestamp	agent.name	rule.description	rule.level	rule.id
Oct 11, 2026 @ 15:54:10.135	DVWA	DVWA login POST req...	5	100510
Oct 11, 2026 @ 15:52:31.509	DVWA	Web server 400 error ...	5	31101
Oct 11, 2026 @ 15:52:31.508	DVWA	Web server 400 error ...	5	31101
Oct 11, 2026 @ 15:52:31.505	DVWA	Web server 400 error ...	5	31101
Oct 11, 2026 @ 15:52:31.503	DVWA	Web server 400 error ...	5	31101
Oct 11, 2026 @ 15:52:31.497	DVWA	Web server 400 error ...	5	31101
Oct 11, 2026 @ 15:52:31.495	DVWA	Web server 400 error ...	5	31101
Oct 11, 2026 @ 15:52:31.492	DVWA	Web server 400 error ...	5	31101
Oct 11, 2026 @ 15:52:31.488	DVWA	Web server 400 error ...	5	31101
Oct 11, 2026 @ 15:46:55.400	DVWA	Web server 400 error ...	5	31101
Oct 11, 2026 @ 15:46:55.396	DVWA	Web server 400 error ...	5	31101
Oct 11, 2026 @ 15:46:55.396	DVWA	Web server 400 error ...	5	31101
Oct 11, 2026 @ 15:46:55.393	DVWA	Web server 400 error ...	5	31101
Oct 11, 2026 @ 15:46:55.390	DVWA	Web server 400 error ...	5	31101
Oct 11, 2026 @ 15:46:55.389	DVWA	Web server 400 error ...	5	31101

Rows per page: 15 < 1 2 >

FIGURE 3.12 – Vue de journaux courants

Lecture de la preuve : Cette liste fournit un contexte de journaux observés dans le laboratoire ; elle doit être comparée aux événements horodatés du test avant toute conclusion.

Chapitre 4

Campagne de tests et résultats

4.1 Reconnaissance et services exposés

Le rapport du 11 octobre décrit un scan Nmap de la cible `192.168.100.30`, détectée active. Les ports TCP 80 et 443 étaient ouverts et 98 ports fermés dans le scan consigné. Le service Web était identifié comme Apache 2.4.52 sur Ubuntu. Netcat a confirmé l'accès aux deux ports Web. La page par défaut Apache était visible et la route `/server-status` répondait HTTP 403 Forbidden. Ces observations établissent la découverte et la disponibilité de services, sans démontrer à elles seules une vulnérabilité exploitable. **[R3, p. 2]**



FIGURE 4.1 – Reconnaissance de la cible avec Nmap
Lecture de la preuve : La capture documente le scan de laboratoire rapporté sur 192.168.100.30 ; le compte rendu identifie HTTP et HTTPS ouverts et 98 ports TCP fermés.

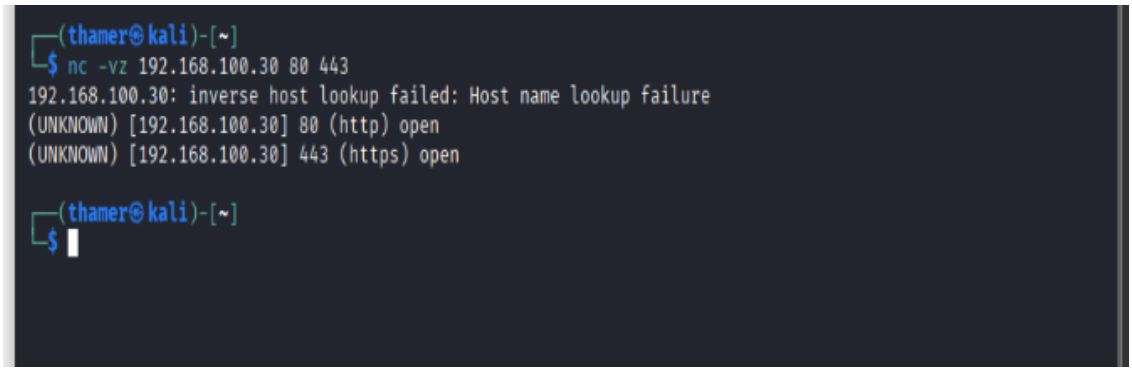


FIGURE 4.2 – Vérification des ports Web avec Netcat

Lecture de la preuve : La capture accompagne la confirmation d'accès aux ports 80 et 443. Une connexion réussie vérifie la joignabilité du service, pas sa sécurité applicative.

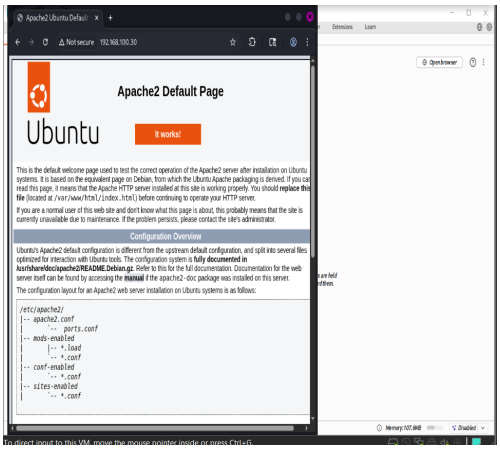


FIGURE 4.3 – Page Apache observée sur la cible

Lecture de la preuve : Cette vue correspond à la page par défaut signalée sur l'hôte Ubuntu et fournit un contexte visuel sur le service Web accessible.

4.2 Accès à DVWA et observation HTTP

DVWA était configuré au niveau de sécurité Low pendant les essais documentés. Les captures montrent l'accès à l'application et à ses modules. Burp Suite a été utilisé pour observer les échanges GET/POST entre le navigateur et la cible. Les preuves établissent l'interception et l'examen de requêtes ; elles n'établissent pas systématiquement le résultat fonctionnel d'une exploitation. **[R3, pp. 2–3]**

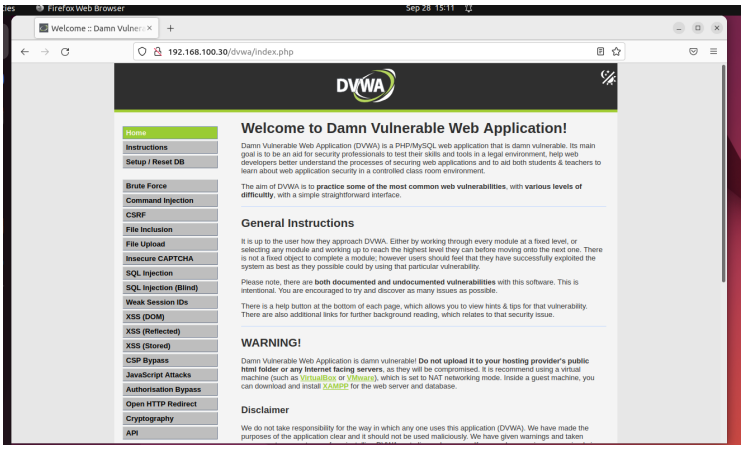


FIGURE 4.4 – Page d'accueil de DVWA

Lecture de la preuve : Cette capture confirme visuellement l'accès à l'application volontairement vulnérable sur le réseau de laboratoire.

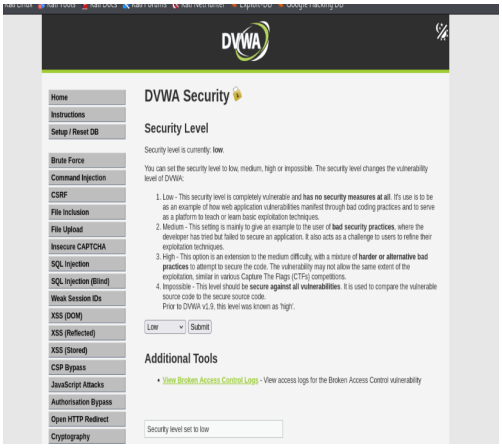


FIGURE 4.5 – Navigation dans les modules DVWA

Lecture de la preuve : La page présente des fonctionnalités pédagogiques, dont celles utilisées pour les scénarios contrôlés.

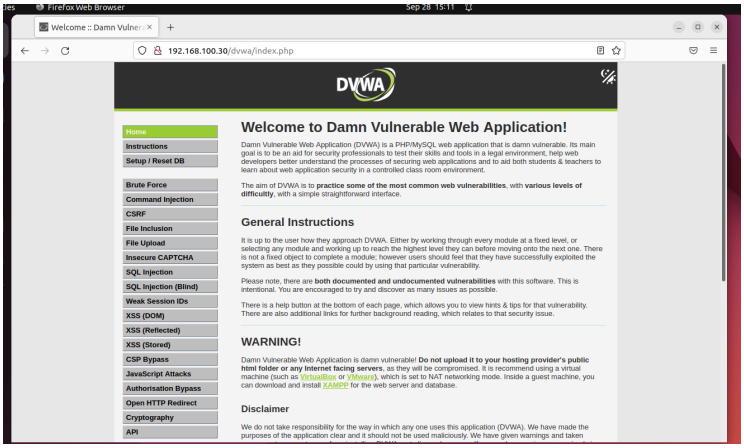


FIGURE 4.6 – Vue complémentaire de l'application DVWA

Lecture de la preuve : Cette image complète la documentation de la cible et des écrans parcourus pendant les essais ; elle n'est pas interprétée comme preuve d'une vulnérabilité réussie.

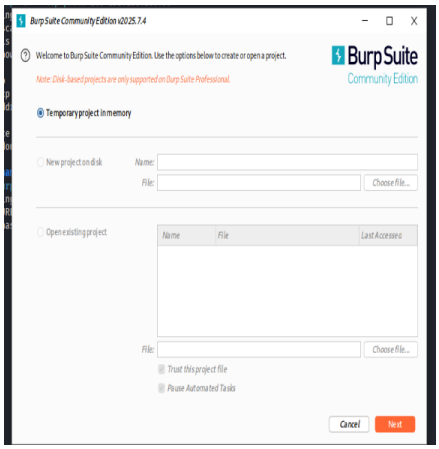


FIGURE 4.7 – Interface Burp Suite Community

Lecture de la preuve : L'outil est utilisé comme proxy pour examiner le trafic HTTP échangé avec DVWA.

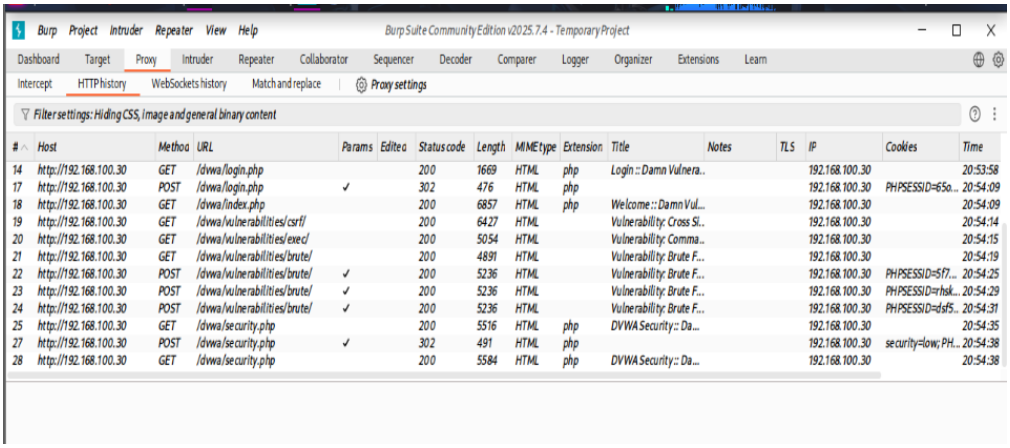


FIGURE 4.8 – Historique de requêtes dans Burp Suite

Lecture de la preuve : L'historique montre des requêtes/réponses observées. Une interception ne prouve pas que des données ont été extraites ou qu'un contrôle a été contourné.

4.3 Essais applicatifs et alertes Wazuh

Le tableau résume les essais explicitement décrits dans le rapport le plus récent. Les identifiants et niveaux sont reproduits comme rapportés ; ils ne sont pas recalculés à partir d'un export brut, qui n'est pas fourni dans le contexte documentaire.

TABLE 4.1: Scénarios DVWA et portée des résultats rapportés

Scénario	**Observation**	**Alerte Wazuh rapportée**	**Conclusion permise**
SQL Injection	Requête de test ; réponse curl HTTP 302.	31103, niveau 7.	Tentative associée à une alerte ; extraction non démontrée.
XSS réfléchi	Requête de test ; réponse curl HTTP 302.	31105, niveau 6.	Événement associé à une alerte ; exécution navigateur non démontrée.
Requêtes répétées DVWA	Cinq requêtes POST de test.	100511, niveau 10 ; 100520, niveau 7 ; 100510, niveau 5.	Activité répétée détectée ; compromission de compte non établie.
Erreur Web	Une règle de journalisation Web figure parmi les résultats.	31101, niveau 5.	Événement observé ; causalité détaillée à confirmer par le journal brut.

La règle 100520 est décrite dans le rapport du 11 octobre comme « Authorization Bypass ». Sa présence dans les résultats mérite une analyse du message brut et de la requête correspondante avant de conclure à un contournement effectif. Les cinq requêtes POST et l'alerte de niveau 10 constituent une détection de comportement répétitif, mais aucune preuve ne montre qu'un mot de passe valide ait été obtenu. **[R3, pp. 2–3]**



FIGURE 4.9 – Requête de test SQL Injection

Lecture de la preuve : Cette preuve accompagne une tentative contrôlée sur DVWA. Le code HTTP 302 rapporté ne démontre pas, à lui seul, une extraction de données.

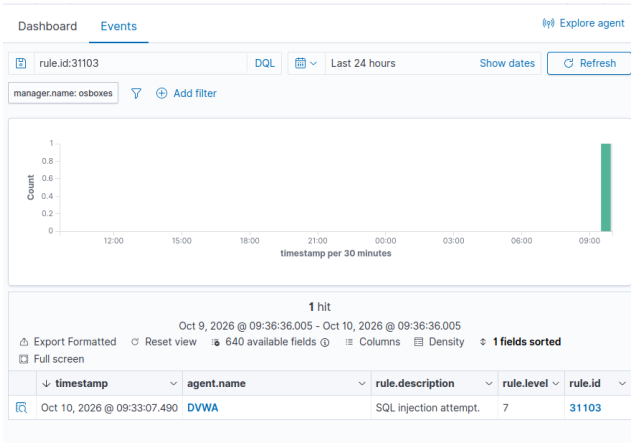


FIGURE 4.10 – Événements Wazuh associés au scénario SQL

Lecture de la preuve : Le rapport associe ce test à la règle 31103 de niveau 7. La correspondance exacte doit idéalement être confirmée par l'horodatage et le journal JSON brut.



FIGURE 4.11 – Requête de test XSS réfléchi

Lecture de la preuve : La capture illustre le scénario testé. Les documents ne montrent pas l'exécution effective du script dans le navigateur.

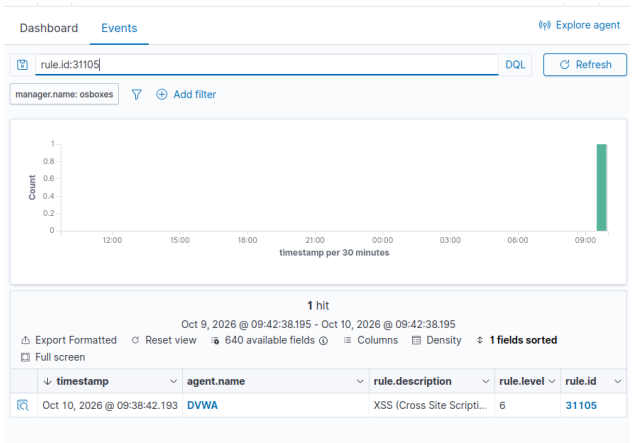


FIGURE 4.12 – Événements Wazuh associés au scénario XSS

Lecture de la preuve : La règle 31105 de niveau 6 est rapportée pour ce test ; la détection ne doit pas être confondue avec une preuve d'exécution côté navigateur.



FIGURE 4.13 – Requêtes d'authentification répétées sur DVWA

Lecture de la preuve : Le rapport décrit cinq requêtes POST de test. Elles concernent le formulaire DVWA, et non une campagne de brute force SSH.

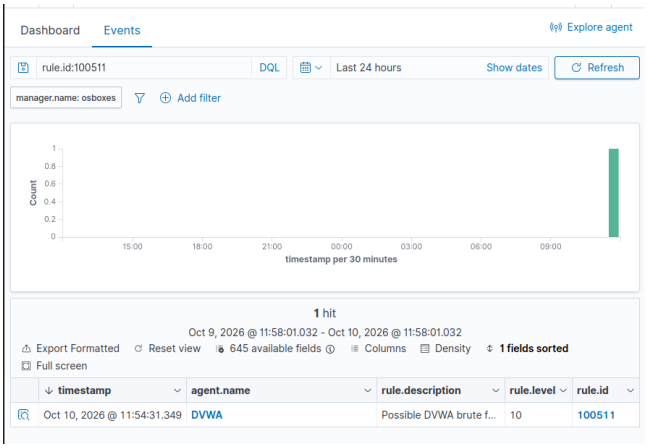


FIGURE 4.14 – Alerte Wazuh sur l'activité répétée

Lecture de la preuve : La règle 100511 de niveau 10 accompagne les requêtes répétées selon le rapport ; aucune compromission de compte n'est prouvée.

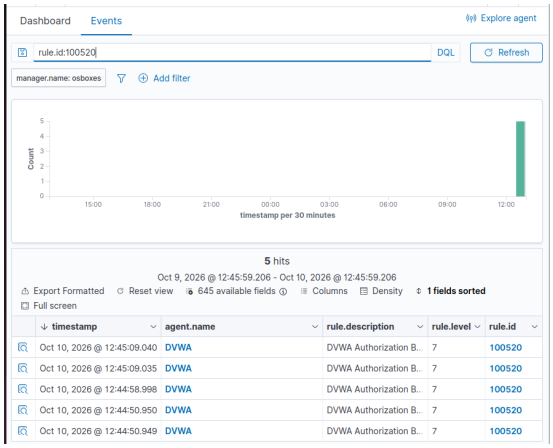


FIGURE 4.15 – Recherche des alertes dans Wazuh

Lecture de la preuve : Cette capture documente la consultation des alertes après les tests applicatifs, sans se substituer aux événements sources horodatés.

4.4 Scénario SSH et analyse de trafic

Les rapports distinguent le scénario SSH initialement prévu, qui envisageait des tentatives répétées, du résultat effectivement documenté. Le 11 octobre, une authentification SSH depuis Kali vers Ubuntu/DVWA a réussi ; le journal indiquait « Accepted password » et Wazuh affichait la règle 5715 « sshd : authentication success », niveau 3. Cette preuve atteste une connexion réussie, mais pas une série d'échecs ni une attaque par brute force SSH. **[R1, p. 3 ; R3, p. 2]**

Wireshark montre l'établissement TCP, l'échange de clés et des paquets SSH chiffrés sur le port 22. Le contenu de la session n'est donc pas lisible dans la capture. Une autre capture concerne le trafic TCP sur le port 80. La capture réseau apporte le contexte transport, tandis que les journaux SSH et l'événement Wazuh renseignent le résultat d'authentification. **[R3, p. 2]**

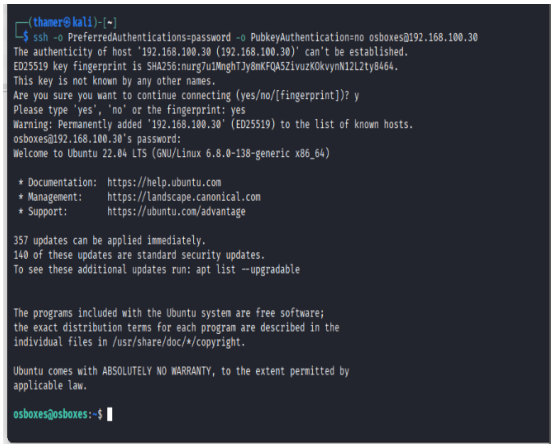


FIGURE 4.16 – Terminal associé au scénario SSH
Lecture de la preuve : L'image documente les commandes de test dirigées vers le laboratoire. Le résultat retenu est celui indiqué dans les journaux : une authentification a réussi.

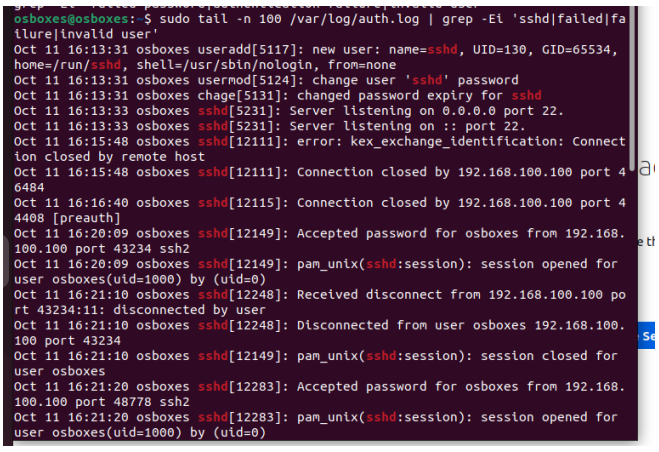


FIGURE 4.17 – Journal SSH de la cible
Lecture de la preuve : Cette capture accompagne l'examen des journaux de la machine Ubuntu pendant la connexion depuis Kali.

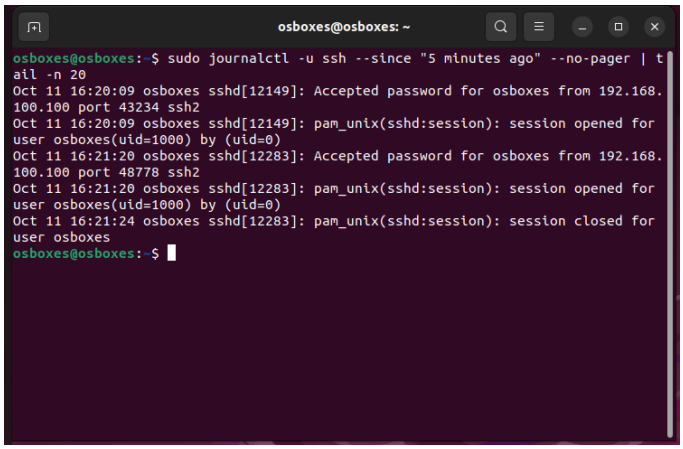


FIGURE 4.18 – Commandes et échanges liés à SSH

Lecture de la preuve : Cette vue complète la trace terminal du scénario et doit être lue avec l'horodatage de l'événement cible.

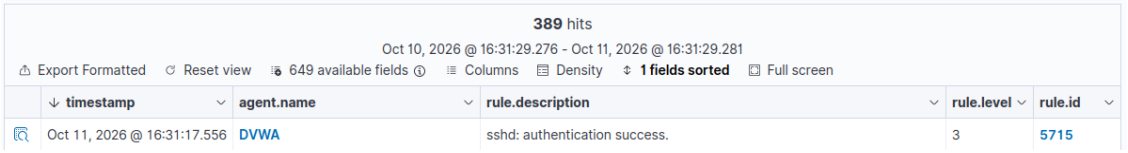


FIGURE 4.19 – Authentification SSH visible dans Wazuh

Lecture de la preuve : Le rapport identifie la règle 5715 et une authentification réussie. Cela ne constitue pas une preuve de brute force SSH.

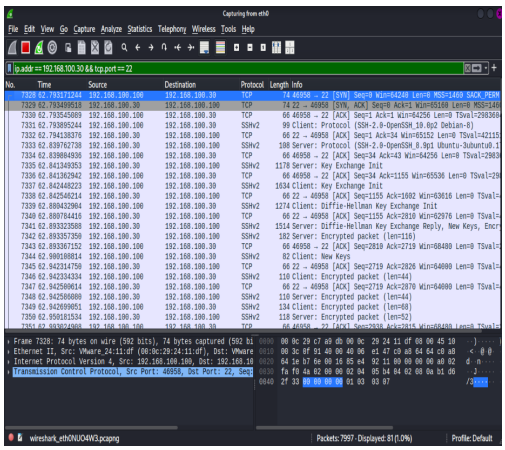


FIGURE 4.20 – Capture Wireshark du trafic SSH

Lecture de la preuve : Wireshark montre la négociation TCP/SSH et des paquets chiffrés sur le port 22 ; le contenu applicatif n'est pas déductible de cette capture.

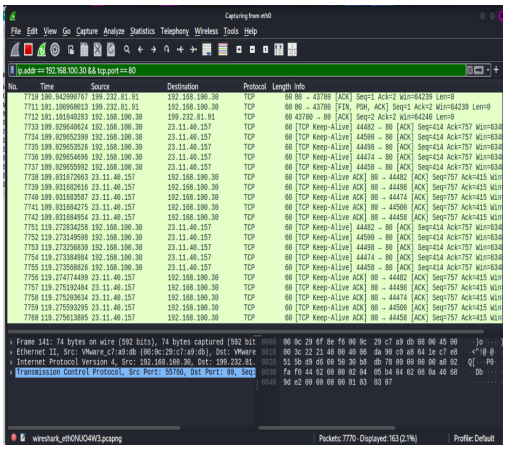


FIGURE 4.21 – Capture Wireshark du trafic HTTP

Lecture de la preuve : Cette vue conserve des échanges TCP associés au port 80 et complète l'observation applicative.

4.5 Scénarios prévus mais non démontrés

Le cahier des charges initial prévoit le FIM : modifier un fichier surveillé, observer l'alerte Wazuh et comparer l'état avant/après. Les rapports fournis ne contiennent pas de preuve que ce test ait abouti. De même, les tests DVWA rapportés couvrent certaines catégories, mais ne constituent pas une validation complète de l'OWASP Top 10. Le projet initial proposait également d'observer la reconnaissance avec Suricata ; les rapports ultérieurs demandent encore de confirmer une alerte réelle jusqu'à Wazuh via la règle 100500. **[R1, pp. 3–5 ; R2, pp. 9, 14–15 ; R3, p. 3]**

Chapitre 5

Orchestration avec Shuffle SOAR

5.1 Déploiement décrit

La VM `192.168.100.40`, initialement prévue pour DFIR-IRIS, a été utilisée pour Shuffle. Le rapport du 28 septembre indique un déploiement Docker Compose et un ajustement des paramètres mémoire d'OpenSearch. Une application Wazuh figurait dans les applications d'organisation Shuffle. Ces éléments décrivent la phase de déploiement ; ils ne garantissent pas la disponibilité actuelle de l'interface. **[R2, pp. 10–11]**

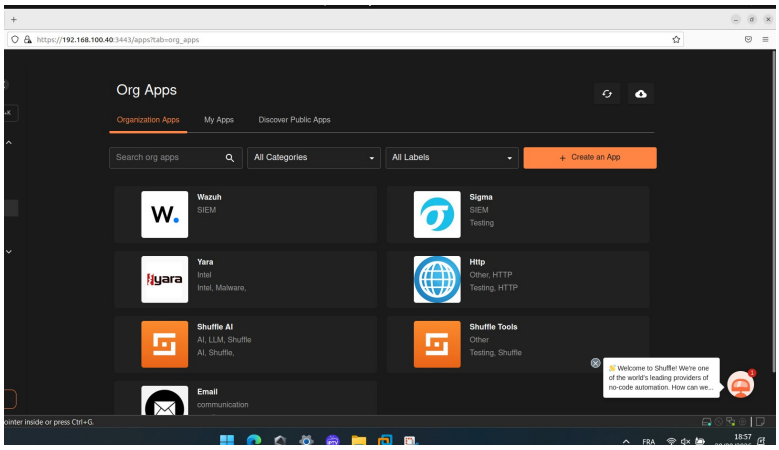


FIGURE 5.1 – Applications disponibles dans Shuffle

Lecture de la preuve : La capture montre la présence historique d'applications dans l'espace Shuffle ; elle ne suffit pas à confirmer que l'organisation est encore accessible.

5.2 Workflow et test du webhook

Le workflow nommé `Wazuh_Suricata_Automation` comportait un déclencheur Webhook et une action de test « Repeat back to me », qui renvoyait le contenu de `$exec`. L'action est adaptée à la vérification du transport, mais ne filtre pas les alertes, ne les enrichit pas et ne réalise aucune mesure de mitigation.

L'intégration Wazuh a été ajoutée dans `ossec.conf` avec le nom `shuffle`, un webhook local en HTTP sur le port 3001 et des données au format JSON. Un premier essai en HTTPS avait échoué en raison de la validation du certificat auto-signé ; le rapport précise que HTTP a été retenu dans

le réseau isolé. Wazuh avait ensuite redémarré correctement. Le recours à HTTP implique que ce trafic n'est pas chiffré : son usage doit rester strictement dans le périmètre isolé et être réévalué si l'architecture change. **[R2, pp. 12–13]**

Un appel de test a retourné un identifiant d'exécution, puis le workflow a été observé avec le statut FINISHED. Un événement système bénin a ensuite généré une alerte Wazuh et un payload a été retrouvé dans une exécution Shuffle. Le rapport décrit notamment les champs `rule_id`, horodatage et texte de l'alerte, avec une alerte 2501. Cela établit une validation historique du chemin Wazuh → webhook Shuffle pour l'événement testé. Cela n'établit ni la branche Suricata réelle, ni un filtrage de sécurité, ni une action défensive. **[R2, pp. 13–15]**

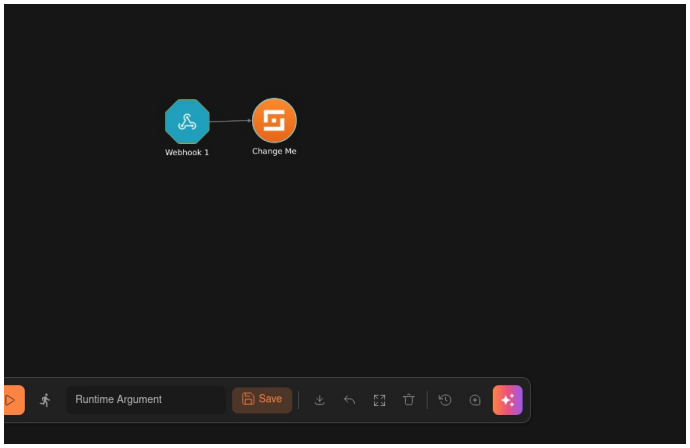


FIGURE 5.2 – Workflow de test Wazuh vers Shuffle
Lecture de la preuve : Le flux représenté contient un déclencheur webhook et une action de répétition de payload, sans décision ni action de défense.

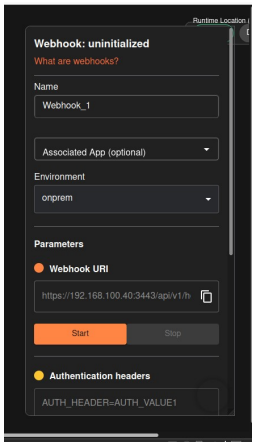


FIGURE 5.3 – État non initialisé du webhook
Lecture de la preuve : Cette capture conserve un état antérieur du point d'entrée avant sa mise en fonctionnement.

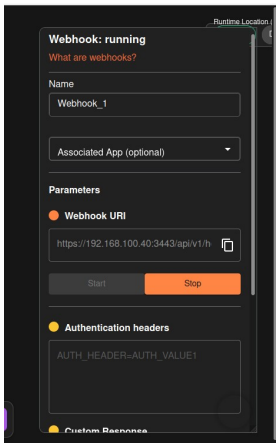


FIGURE 5.4 – Webhook en état de fonctionnement lors d'un test
Lecture de la preuve : La capture montre un état « running » rapporté historiquement ; elle ne certifie pas son statut à la date du diagnostic ultérieur.

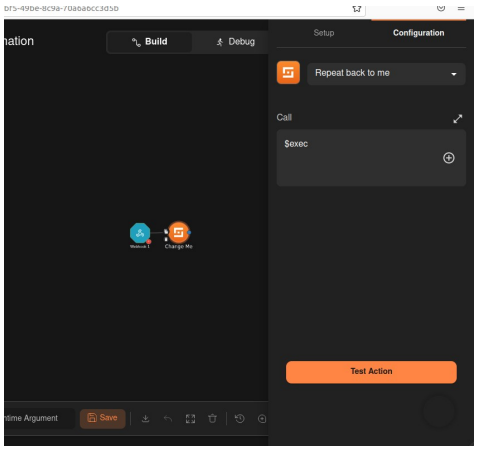


FIGURE 5.5 – Action de retour configurée dans Shuffle
Lecture de la preuve : L'action restitue le payload reçu afin de vérifier son passage dans le workflow ; elle n'effectue pas de remédiation.

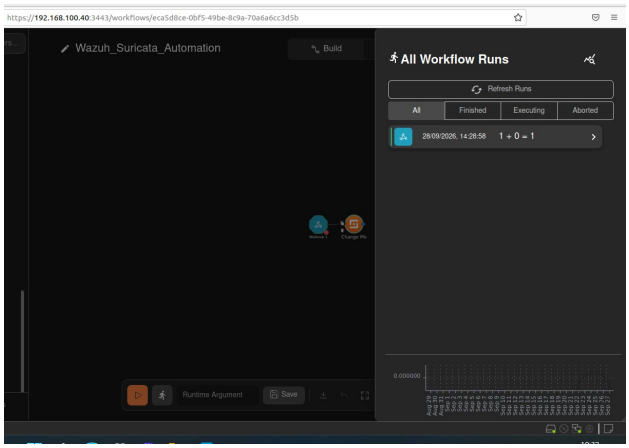


FIGURE 5.6 – Historique d'exécutions du workflow
Lecture de la preuve : L'historique constitue une preuve visuelle d'exécutions de test passées et non d'une réponse défensive actuelle.

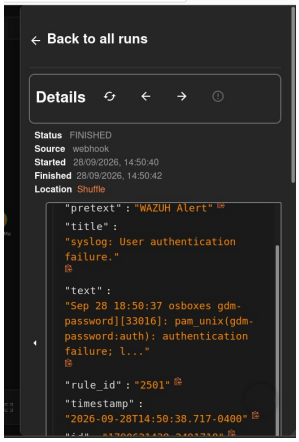


FIGURE 5.7 – Détail d'une exécution et de son payload

Lecture de la preuve : La vue accompagne la validation historique d'un payload reçu dans le webhook et consultable via l'exécution.

5.3 État le plus récent et réserve de validation

Le rapport du 11 octobre indique que les conteneurs principaux étaient visibles comme actifs, que le frontend répondait HTTP 200 sur le port 3001, que le backend répondait sur le port 5001 et que les tests d'API non authentifiés renvoyaient 401. OpenSearch renvoyait également 401 sans identifiants. Cependant, l'interface affichait « Failed getting org » ; l'accès à l'organisation et l'exécution actuelle du workflow n'étaient donc pas confirmés. Aucune action défensive automatique n'est documentée. **[R3, pp. 2–3]**

En conséquence, le statut consolidé est : **transport Wazuh–Shuffle démontré historiquement ; disponibilité et exécution actuelles à revalider ; automatisation de réponse non démontrée**. La différence entre les rapports est chronologique : le résultat positif du 28 septembre est conservé, mais le diagnostic plus récent prévaut pour qualifier l'état à la date de consolidation.

5.4 Cible fonctionnelle recommandée

La suite du projet peut transformer le workflow de test en chaîne contrôlée en ajoutant les fonctions suivantes, dans cet ordre :

1. réparer et documenter l'accès authentifié à l'organisation Shuffle ;
2. vérifier qu'un seul événement bénin atteint le webhook et conserver payload, horodatage, exécution et statut ;
3. filtrer les alertes pertinentes, notamment celles du groupe Suricata, plutôt que de transmettre indifféremment tout événement au-dessus d'un niveau ;
4. enrichir le message avec l'adresse source, la destination, la signature, la sévérité et l'horodatage ;
5. commencer par une notification ou la création d'un cas, sans blocage automatique ;

6. n'envisager une action de confinement qu'après une validation manuelle et des essais répétables, isolés et réversibles.

Chapitre 6

Application AI SOC Analyst

6.1 Fonctionnement observé

AI SOC Analyst est décrite comme une application Flask locale, accessible sur `127.0.0.1:5000`. L'utilisateur sélectionne manuellement un export JSON issu de Wazuh ; l'application traite les données et peut générer un document PDF. Ce dispositif est un outil d'analyse hors ligne par import manuel, distinct du flux d'alertes Wazuh → Shuffle.

Les captures et le rapport du 11 octobre indiquent qu'un export de 208 alertes a été traité. Les statistiques visibles dans un rapport local répartissent les événements en 205 pour l'agent « osboxes » et 3 pour « DVWA ». Le rapport précise que Gemini était indisponible lors de cette exécution et que la sortie correspondait à des statistiques locales. Une autre sortie affiche le libellé de modèle « Gemini 3.8 Flash » ; cette mention est rapportée telle qu'elle apparaît dans la preuve, sans validation indépendante du modèle réellement appelé. **[R3, p. 2]**

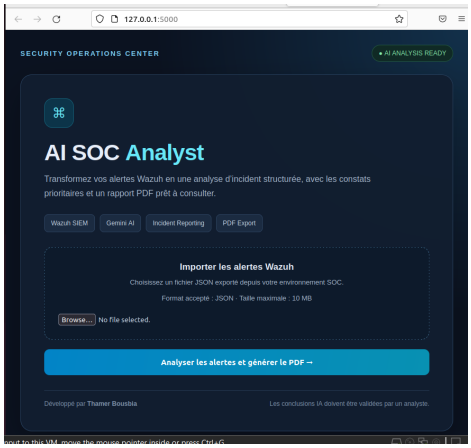


FIGURE 6.1 – Interface locale AI SOC Analyst

Lecture de la preuve : La capture présente l'interface utilisée pour sélectionner un export d'alertes et lancer le traitement local.

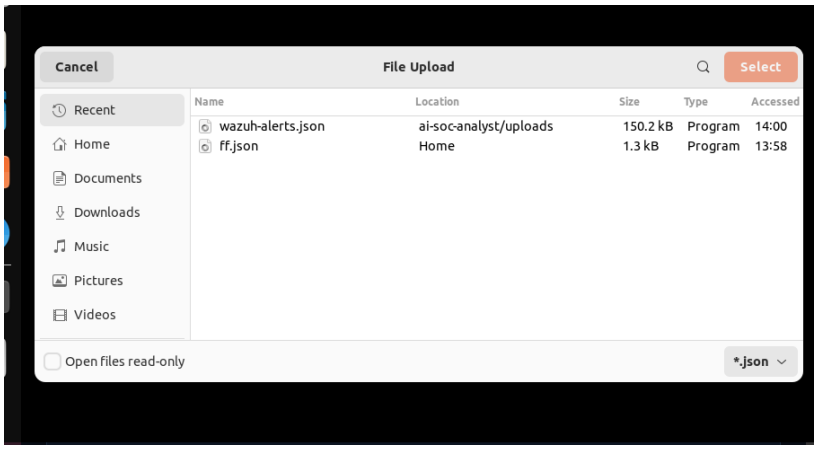


FIGURE 6.2 – Sélection du fichier à analyser

Lecture de la preuve : Cette étape illustre l'import manuel d'un fichier ; elle ne représente pas une collecte automatique depuis Wazuh.

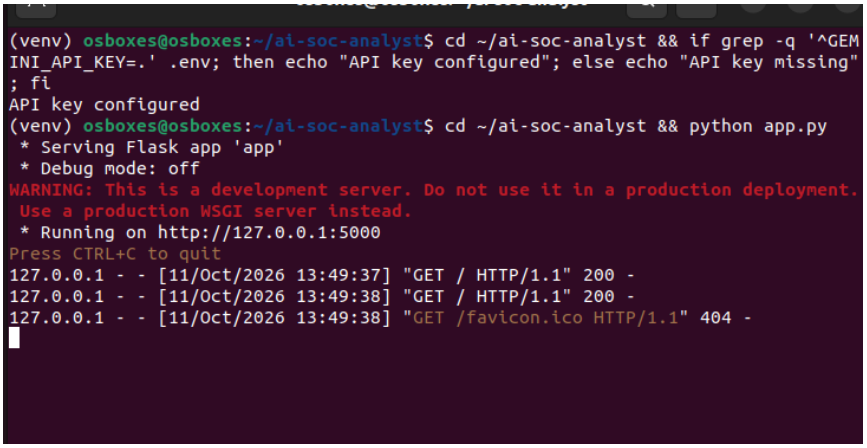


FIGURE 6.3 – Console de l'application Flask

Lecture de la preuve : La console accompagne le démarrage et les requêtes vers l'application locale.

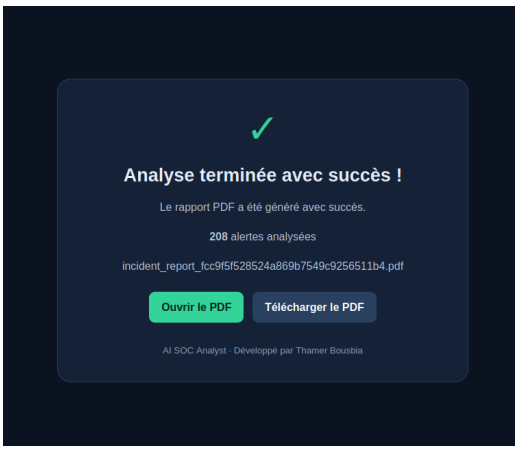


FIGURE 6.4 – Confirmation du traitement des alertes

Lecture de la preuve : La preuve accompagne le traitement rapporté d'un export comprenant 208 alertes et la création d'un rapport.

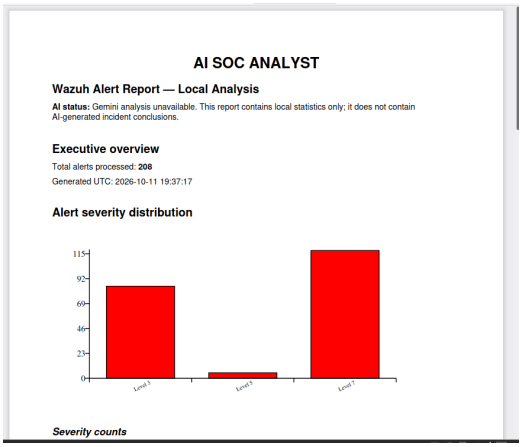


FIGURE 6.5 – Exemple de rapport généré

Lecture de la preuve : La capture montre un document issu de l'analyse. Ses conclusions doivent être confrontées aux événements Wazuh bruts avant d'être réutilisées.

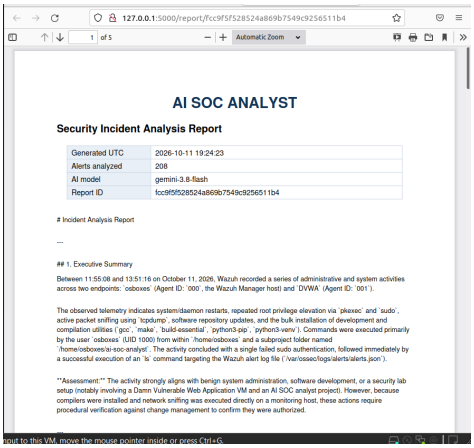


FIGURE 6.6 – Rapport complémentaire avec statistiques

Lecture de la preuve : Cette vue présente une synthèse locale de l'export et ne doit pas être assimilée à une analyse par modèle lorsque Gemini est indisponible.

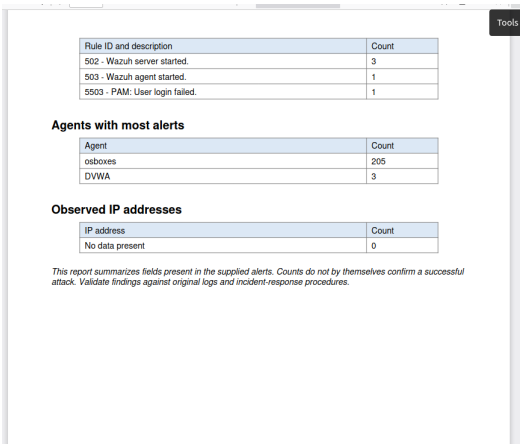


FIGURE 6.7 – Vue complémentaire du rapport d'analyse

Lecture de la preuve : Cette page conserve une autre sortie documentaire. Le mode utilisé doit être explicitement indiqué avec chaque rapport produit.

6.2 Limites d'interprétation

Le volume importé (208 alertes) prouve que l'application a accepté et traité un export dans l'exécution documentée ; il ne garantit ni exhaustivité de la collecte ni exactitude des conclusions. Les rapports doivent distinguer au minimum le nombre d'événements importés, la période couverte, les filtres appliqués, l'état de l'API IA et la nature statistique ou générative du résultat. Une analyse assistée ne remplace pas la vérification de l'événement brut, de la règle et de l'agent source.

Chapitre 7

Évaluation globale et recommandations

7.1 Bilan de validation

La qualification ci-dessous tient compte à la fois du statut historique rapporté en septembre et du diagnostic plus récent du 11 octobre. « Démontré » signifie que les sources fournies décrivent une observation ou une preuve ; cela ne garantit pas que le composant fonctionne encore sans nouveau test.

Table 7.1: Matrice de validation consolidée au 12 octobre 2026

Élément	**Statut**	**Justification et réserve**
Topologie VMware et rôles VM	Documenté	Adresses et fonctions décrites pour le réseau 192.168.100.0/24 ; la topologie initiale 192.168.56.0/24 reste historique.
Reconnaissance Nmap / Netcat	Démontré dans les rapports	Cible .30 observée ; ports Web 80/443 accessibles dans les tests décrits.
Apache et DVWA	Démontré dans les rapports	Page Web et modules DVWA accessibles ; DVWA utilisé en niveau Low lors des tests.
Inspection Burp	Démontré	Requêtes HTTP observées ; réussite d'exploitation non déduite de l'interception.
Alertes Wazuh DVWA	Démontré avec réserves	Plusieurs ID de règles rapportés ; corrélation exacte à confirmer à partir des journaux bruts.
Connexion SSH réussie	Démontré	Journal « Accepted password » et règle 5715 ; ne prouve pas une attaque par brute force SSH.
Capture Wireshark	Démontré	Échanges TCP/SSH et Web visibles ; contenu SSH chiffré.
Réception syslog Suricata/Wazuh	Démontrée historiquement	Paquets UDP/514 et événements EVE observés dans le rapport du 28 septembre ; à recontrôler après les changements.

Élément	Statut	Justification et réserve
Suricata → règle Wazuh 100500	À revalider	Décodeur testé, mais alerte réelle de bout en bout non confirmée dans le rapport récent.
Webhook Wazuh → Shuffle	Démontré historiquement ; état actuel incertain	Une exécution FINISHED et une alerte reçue sont rapportées en septembre ; erreur « Failed getting org » en octobre.
Action de réponse SOAR	Non démontrée	Le workflow illustré répète le payload ; aucun blocage ou autre réponse défensive vérifiée.
AI SOC Analyst	Partiellement démontré	Import manuel de 208 alertes et génération de PDF ; disponibilité et mode Gemini différents selon les sorties.
FIM	Non démontré	Scénario prévu dans le cadrage ; aucune preuve d'un test final fourni.
Couverture OWASP complète	Non démontrée	Quelques modules DVWA testés ; aucune matrice complète des catégories n'est fournie.

7.2 Limites et risques

— **Évolution de configuration :** les plans d'adressage `192.168.56.0/24` et `192.168.100.0/24` correspondent à des phases distinctes. Chaque nouvelle preuve doit mentionner le réseau et la date.

— **Causalité des alertes :** l'apparition d'une règle autour d'un test ne suffit pas à prouver la causalité sans agent, heure, requête et événement brut concordants.

— **Portée des tests :** les réponses 302, les captures de requêtes et les alertes ne constituent pas une preuve d'extraction SQL, d'exécution XSS ou de compromission de compte.

— **Confidentialité des captures :** les flux SSH visibles dans Wireshark sont chiffrés ; il ne faut pas attribuer à la capture le contenu de la session.

— **Transport webhook :** le HTTP local retenu pour contourner un certificat auto-signé n'est pas chiffré et ne doit pas être exposé hors réseau isolé.

— **Automatisation :** envoyer toutes les alertes vers une action de blocage sans filtrage ni validation créerait un risque de faux positifs et d'interruption de service.

— **Analyse IA :** séparer explicitement les sorties statistiques locales des sorties générées par un modèle et vérifier toute recommandation sur les journaux sources.

7.3 Plan d'action priorisé

TABLE 7.2: Travaux recommandés et preuves attendues

Priorité	Action	Critère de clôture
1	Stabiliser le laboratoire	Archiver adresses, versions, routes, états des services et règles actives ; sauvegarder les configurations.
2	Rétablir Shuffle	Corriger « Failed getting org », vérifier l'accès authentifié et conserver le résultat horodaté d'un webhook bénin.
3	Valider Suricata vers Wazuh	Produire un événement de test autorisé, conserver l'EVE JSON, confirmer le syslog, le décodage, la règle 100500 et l'alerte Wazuh correspondante.
4	Valider le chemin SOAR	Relier l'alerte au payload et à l'identifiant d'exécution Shuffle ; commencer par une notification ou un cas, sans action destructive.
5	Réaliser le test FIM	Choisir un fichier de laboratoire, archiver son état avant/après et montrer l'événement Wazuh produit.
6	Formaliser les tests Web	Documenter pour chaque test la requête, la réponse, l'agent, la règle, l'horodatage et la limite d'interprétation.
7	Qualifier AI SOC Analyst	Indiquer période, filtres, nombre d'alertes, mode local/IA et modèle réellement sollicité ; valider les conclusions manuellement.

7.4 Protocole de validation reproductible

Le protocole suivant est une proposition pour les essais futurs et ne décrit pas des tests déjà réalisés :

1. confirmer que les VM concernées sont isolées et que la cible est bien `192.168.100.30` ; relever la date, l'heure et la configuration active ;
2. choisir un seul scénario bénin et autorisé, puis enregistrer le poste source, la destination et la fenêtre temporelle ;
3. capturer le trafic sur l'interface pertinente sans inclure de réseau externe ;
4. vérifier les journaux locaux de la cible ou l'événement EVE attendu avant de rechercher l'alerte dans Wazuh ;
5. exporter l'alerte JSON et consigner agent, règle, niveau, champs source/destination et horodatage ;
6. si le test concerne Shuffle, confirmer l'accès à l'organisation, le payload reçu, l'identifiant

d'exécution et son statut ;

7. commencer par une notification non destructive, puis répéter le test après chaque correction ;
8. archiver ensemble les PCAP, événements bruts, captures et configuration ; annoter toute différence entre exécutions.

Conclusion générale

Le projet a abouti à un laboratoire virtualisé articulé autour de Kali, d'une cible Ubuntu/DVWA, de pfSense avec Suricata, de Wazuh et d'une plateforme Shuffle. Les documents démontrent la reconnaissance et l'accessibilité des services Web, l'observation de requêtes DVWA avec Burp Suite, plusieurs alertes Wazuh liées à des essais applicatifs, une authentification SSH réussie, des captures Wireshark et la génération locale de rapports à partir d'un export Wazuh de 208 alertes.

Le rapport de déploiement du 28 septembre décrit aussi la réception d'un événement Wazuh par un webhook Shuffle et la visibilité de trafic EVE/syslog dans la chaîne. Ces résultats sont conservés comme validations historiques. Ils ne suffisent toutefois pas à affirmer que la configuration est restée identique après reconfiguration : le rapport du 11 octobre signale « Failed getting org » et ne confirme pas l'accès actuel au workflow. La règle Suricata 100500 n'est pas associée à une alerte réelle de bout en bout dans les preuves les plus récentes ; aucune action SOAR défensive, aucun test FIM abouti et aucune couverture OWASP complète ne sont établis.

Le résultat doit donc être présenté comme un **prototype SOC/SOAR prometteur et partiellement validé**. La prochaine démonstration décisive consistera à produire, dans le réseau isolé, une trace datée reliant un test contrôlé à l'événement EVE, à l'alerte Wazuh puis à une exécution Shuffle actuelle, avant d'ajouter une notification sûre, traçable et répétable.

Sources documentaires

R1 *Laboratoire de détection des cybermenaces – Projet 1 : architecture, rôles, scénarios d'attaque, détection et défense*. Rapport technique de cadrage, septembre 2026, 5 pages.

R2 *Laboratoire de détection et d'orchestration des cybermenaces : pfSense / Suricata → Wazuh → Shuffle SOAR*. Rapport de déploiement technique, 28 septembre 2026, 15 pages.

R3 *Rapport SOC/SOAR – Laboratoire de détection et d'analyse des cybermenaces*. État des validations et résultats, 11 octobre 2026, 7 pages.

Les références de page indiquées dans le texte renvoient à la pagination des documents originaux. Les captures conservées dans le dossier `art/` sont intégrées comme figures aux chapitres correspondants ; elles ne sont pas ajoutées en galerie séparée.

Chapitre A

Correspondance entre rapports sources et rapport consolidé

Table A.1: Traçabilité des principaux apports des trois rapports

Thème	Rapport source	Intégration et réserve retenue
Cahier des charges, architecture initiale et calendrier	R1, pp. 2–5	Repris dans les chapitres 1 et 2 comme conception initiale ; réseau 192.168.56.0/24 conservé comme historique.
Inventaire VMware et réseau déployé	R2, pp. 3–5	Repris au chapitre 2 ; réseau principal documenté 192.168.100.0/24.
pfSense, Suricata et syslog/EVE	R2, pp. 5–7	Repris au chapitre 3 ; réception historique constatée, alerte réelle 100500 à revalider selon R3.
Wazuh et règles hôte	R2, pp. 7–9	Repris au chapitre 3, avec agent, règle et contexte afin d'éviter de relier les événements sans preuve.
DVWA et agent	R2, pp. 9–10 ; R3, pp. 2–3	Repris aux chapitres 2 et 4 ; tests Low et portée limitée des résultats explicités.
Webhook Wazuh–Shuffle	R2, pp. 10–15 ; R3, pp. 2–3	Validation historique de transport conservée ; état plus récent incertain après « Failed getting org ».
Nmap, Netcat, Burp et tests DVWA	R3, pp. 2–3	Repris au chapitre 4 ; les redirections et alertes ne sont pas surinterprétées comme compromissions.
SSH et Wireshark	R3, p. 2	Repris au chapitre 4 ; connexion réussie distinguée du brute force SSH ; chiffrement signalé.

Thème	**Rapport source**	**Intégration et réserve retenue**
AI SOC Analyst	R3, p. 2	Repris au chapitre 6 ; import manuel, 208 alertes, mode local/Gemini qualifié selon la preuve.
FIM et couverture OWASP	R1, pp. 3–5 ; R3, p. 3	Conservés comme objectifs ou travaux restant à démontrer, pas comme résultats achevés.

Chapitre B

Matrice des critères de réception

Les critères ci-dessous sont des conditions proposées pour la clôture technique du laboratoire. Ils ne modifient pas les statuts de validation observés à la date de consolidation.

Table B.1: Critères de réception proposés

Composant	**Preuve attendue**	**État au 12 octobre 2026**
Suricata	Événement EVE JSON lié à un scénario autorisé	Télémétrie historique ; alerte réelle à reproduire.
Wazuh réseau	Réception syslog, décodage, règle 100500 et alerte liée	À revalider après les modifications.
Shuffle	Payload attendu et exécution actuelle terminée	Validé historiquement ; organisation actuelle à réparer/vérifier.
Réponse SOAR	Notification ou cas non destructif traçable	Non démontrée dans les sources.
FIM	Modification contrôlée, état avant/après et alerte	Non démontré dans les sources.
AI SOC Analyst	Export, période, mode et résultats reproductibles	Import de 208 alertes démontré ; nature des sorties à préciser.

Chapitre C

Répertoire et conservation des preuves

Les 49 captures du dossier `art/` sont intégrées dans les chapitres 2 à 6, au voisinage des analyses qu'elles illustrent. Trois captures de Suricata provenant du rapport de déploiement ont été ajoutées aux preuves du projet. La liste des figures générée en début de rapport sert d'index ; chaque capture est utilisée une seule fois. Les images constituent des preuves visuelles de l'interface au moment de la capture et ne remplacent ni les journaux bruts ni les configurations exportées.

Pour rendre les prochains essais reproductibles, conserver un dossier de preuve distinct pour chaque scénario et associer aux captures les éléments techniques suivants :

— un identifiant de test, sa date, son heure et le fuseau horaire utilisé ;

— les adresses source/destination, le nom de l'agent et les interfaces réseau concernées ;

— les versions des VM et les configurations actives de pfSense, Suricata, Wazuh et Shuffle ;

— la requête ou l'action de test, le résultat fonctionnel attendu et le résultat observé ;

— le fichier PCAP limité à la fenêtre de test, l'événement EVE JSON et l'alerte Wazuh exportée ;

— pour une exécution SOAR, le payload reçu, l'identifiant d'exécution, le statut et le résultat de l'action.

TABLE C.1 – Éléments à archiver selon le type de validation

Validation	**Fichiers ou valeurs**	**Point de contrôle**
Reconnaissance	Résultat Nmap, hôte ciblé, plage et date	Vérifier que seule la cible du laboratoire a été incluse.
Test Web DVWA	Requête/réponse, capture Burp, règle Wazuh, agent	Relier les événements au même test et éviter de déduire une exploitation sans preuve fonctionnelle.
Inspection réseau	PCAP, EVE JSON, interface et filtre de capture	Séparer télémétrie réseau et alertes Suricata.
Transfert SIEM/SOAR	Export JSON Wazuh, payload webhook, identifiant d'exécution	Vérifier le même horodatage et le statut final du workflow.
FIM	Empreinte/état avant-après, alerte et chemin du fichier	Utiliser un fichier de laboratoire et conserver la preuve du retour à l'état initial.